# Data Privacy - Project 4 Discussion

Hoang Nguyen

October 2025

## 1 Introduction

For this project, I implemented three different schemes for secure multiparty computation, including no protection, additively homomorphic Paillier, Shamir Secret Sharing and a Differential Privacy mechanism. I then run the experiment with five different values of $n$ and plot the runtime results using line graphs. I also extract the time into a different CSV file in order to test different settings.

## 2 Discussion

I chose the values $n = 20, 100, 500, 1000$, and 1500 to test how each scheme scales with the number of participants or input values. The smaller values (20 and 100) show how the methods perform with small data sizes, while the larger values (500 and 1000 and 1500) show how runtime increases as the input grows. This range gives a good balance between quick experiments and visible performance differences, and it helps reveal whether each scheme's runtime grows linearly or exponentially with $n$.
From the graph(below), we can see that the y-axis is on a log scale. I used this scale to show the results for the DP and the no protection scheme more clearly. The Shamir and Paillier scheme take much longer to run, which makes the other two lines hard to see on a normal (linear) scale.
When running the code, I noticed that the results can change a little between runs. To make the data more accurate, I ran each scheme three times for every value of $n$ and then took the average of those times. This gives a better idea of the real runtime for each scheme.
For the no protection scheme, the code only needs to calculate the average of the numbers. This is a very simple task and does not take any noticeable time, so there is not much to discuss about this part.
For the Paillier scheme, I used the `phe` package, which made it much easier to set up. The encryption, decryption, computation and total runtime for Paillier is larger for smaller values of $n$. As $n$ increases the growth rate of this scheme is slower compared to Shamir.
For the Shamir scheme, for smaller value of $n$ the algorithm runs quite fast, nearly as fast as the plain average. However, the growth rate for this scheme is fast, thus, as $n$ gets larger, the total runtime, encryption, decryption, and computation is the slowest out of the four schemes.
For DP mechanism, the runtime is quite fast, nearly as fast as plain average, since this does not involve any computation with prime number. However, for the accuracy compared with the true average of the data is lower than the other three schemes since this scheme adds a laplacian noise to the data.
In summary, the no protection scheme, and DP mechanism run the fastest and almost instantly. The Paillier scheme runs slower but still fast overall. The Shamir scheme is the slowest because it uses heavy modular exponentiation with very large numbers. For the accuracy analysis, the other three methods is 0 as desired and the number for DP fluctate with in a certain range.

## 3 Graph

Below is the graph that I produced using the running time collected from running the experiment

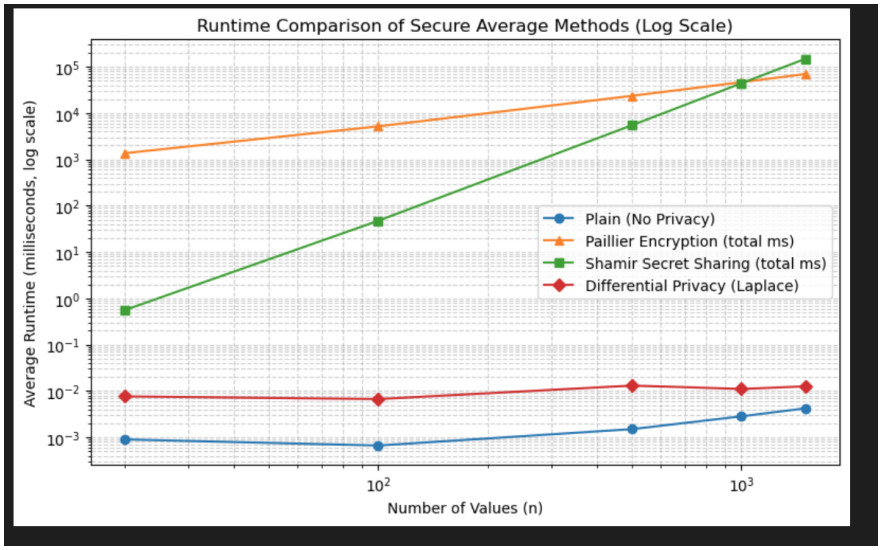


Figure 1: Total Runtime comparison of Plain, Shamir's Secret Sharing, Paillier Encryption, and DP (log scale).

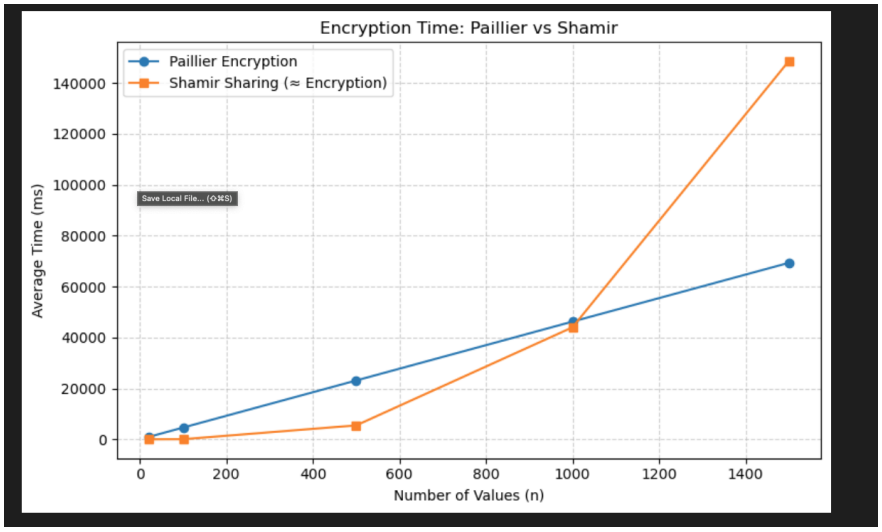


Figure 2: Encryption Runtime comparison of Shamir's Secret Sharing, and Paillier Encryption (ms).

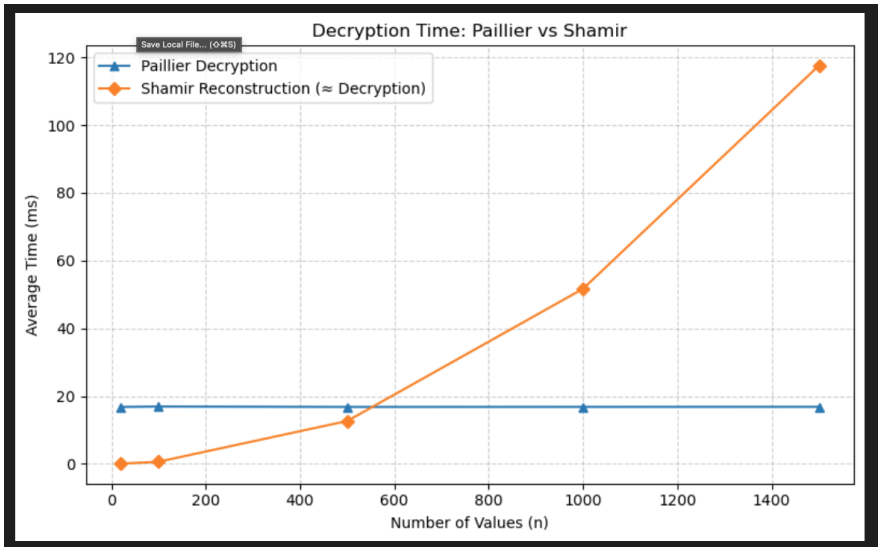


Figure 3: Decryption Runtime comparison of Shamir's Secret Sharing, and Paillier Encryption (ms).

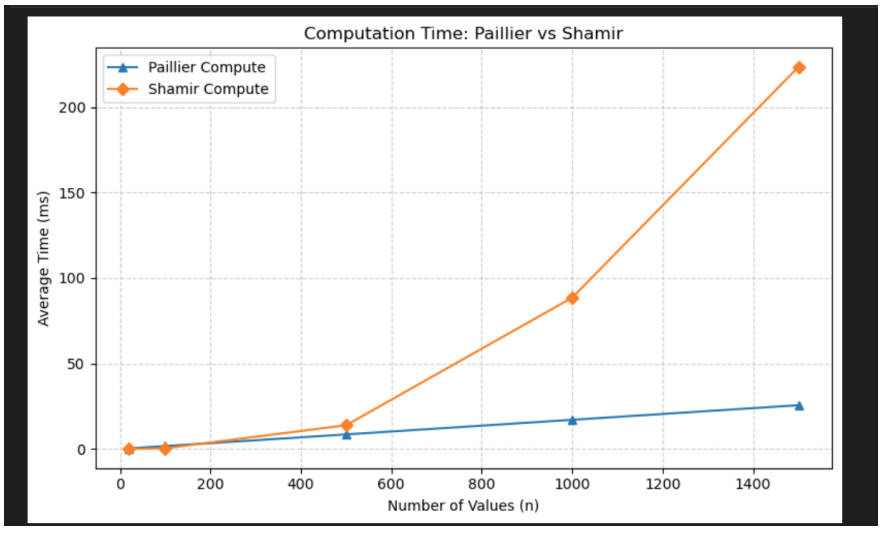


Figure 4: Computation Runtime comparison of Shamir's Secret Sharing, and Paillier Encryption (ms).

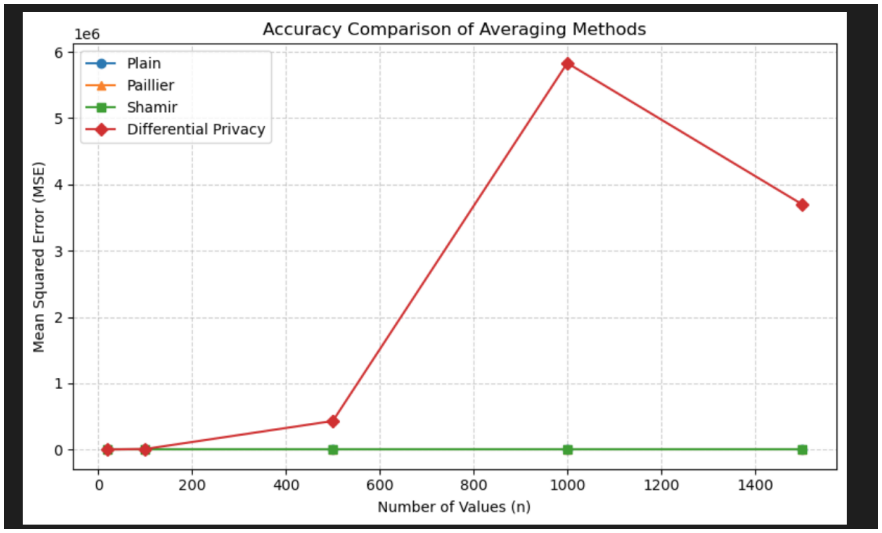


Figure 5: Accuracy Analysis of Plain, Shamir's Secret Sharing, Paillier Encryption, DP (MSE).